MACHINE LEARNING

>:_ FACULTAD DE INGENIERÍA

Universidad Rafael Landívar
Identidad Jesuita en Guatemala

POSGRADOS LANDÍVAR

Laboratorio 3.2

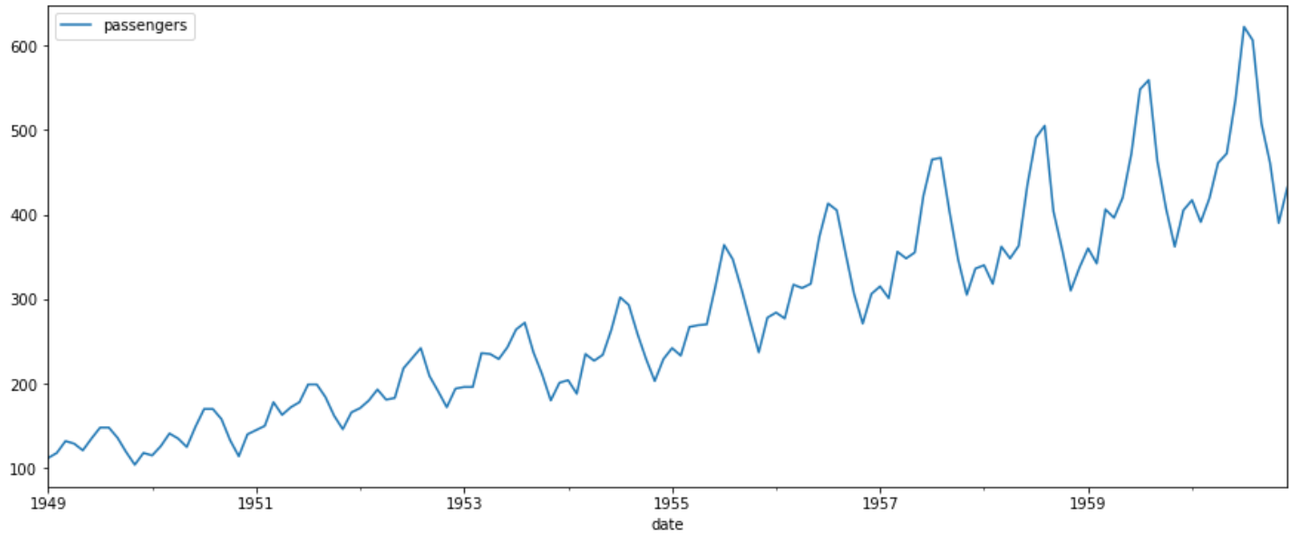
Time Series Analisis

In [1]:

```
from datetime import datetime
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.statespace.sarimax import SARIMAX
import numpy as np
```

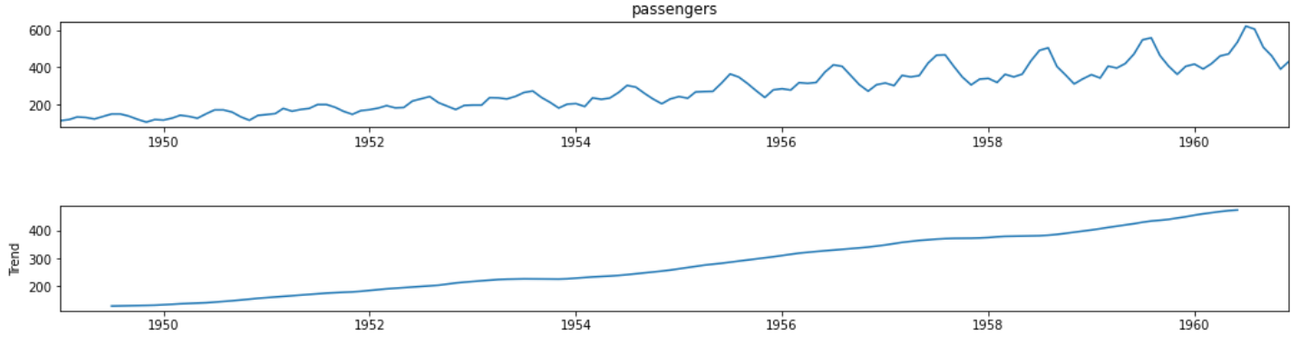
In [2]:

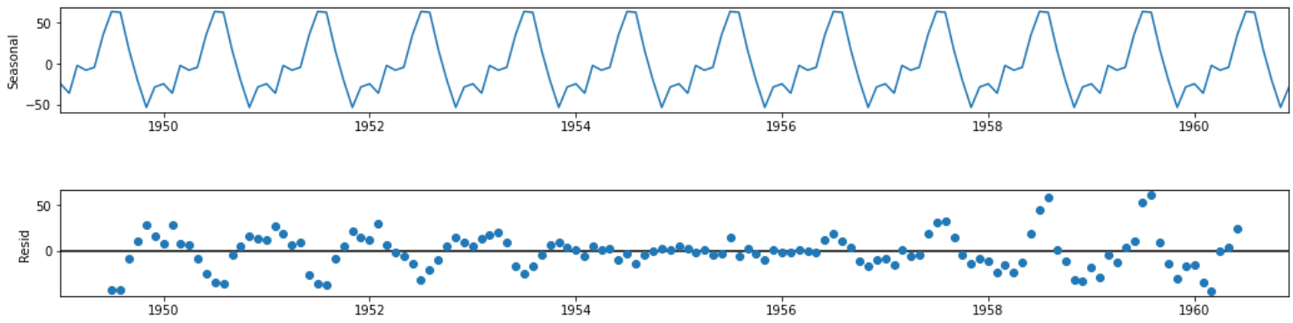
```
data = pd.read_csv('air_passengers.csv',sep=',')
df = pd.DataFrame(data, columns = ['date', 'passengers'])

# Set the Date as Index
df['date'] = pd.to_datetime(df['date'])
df.index = df['date']
del df['date']


df.plot(figsize=(15, 6))
plt.show()
```



In [3]:

```
decompose = seasonal_decompose(df['passengers'],model='additive', period=12)

decompose_graph = decompose.plot()
decompose_graph.set_size_inches((16, 9))
plt.show()
```



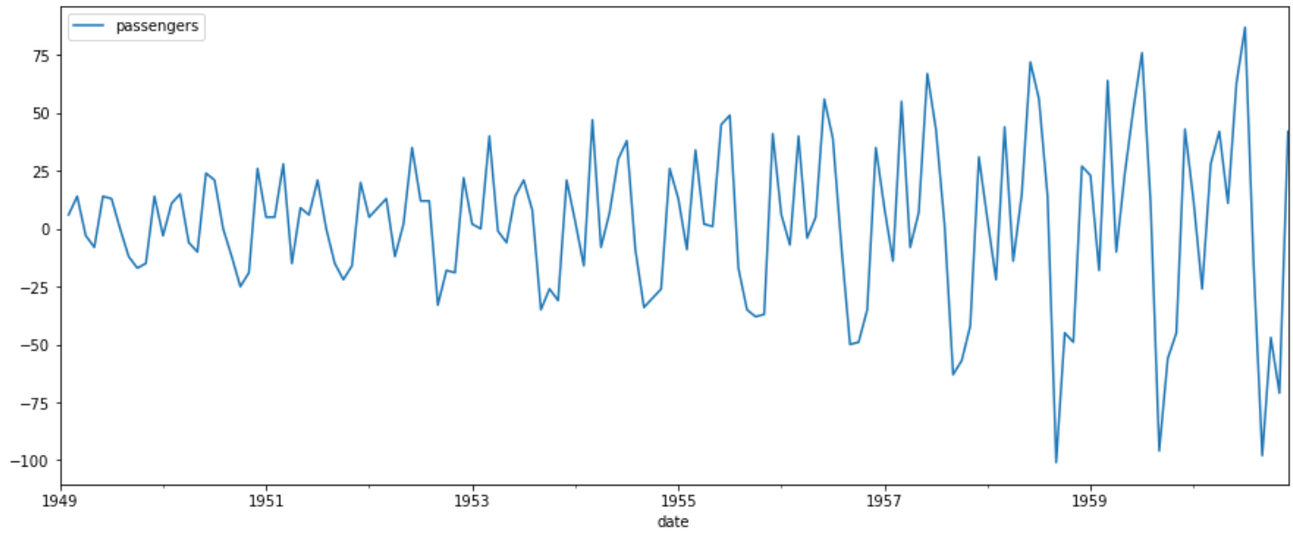


Diferenciación

Utilizaremos diferenciación para volver estacionaria nuestra serie de tiempo. ¿Nota algo raro?

In [4]:

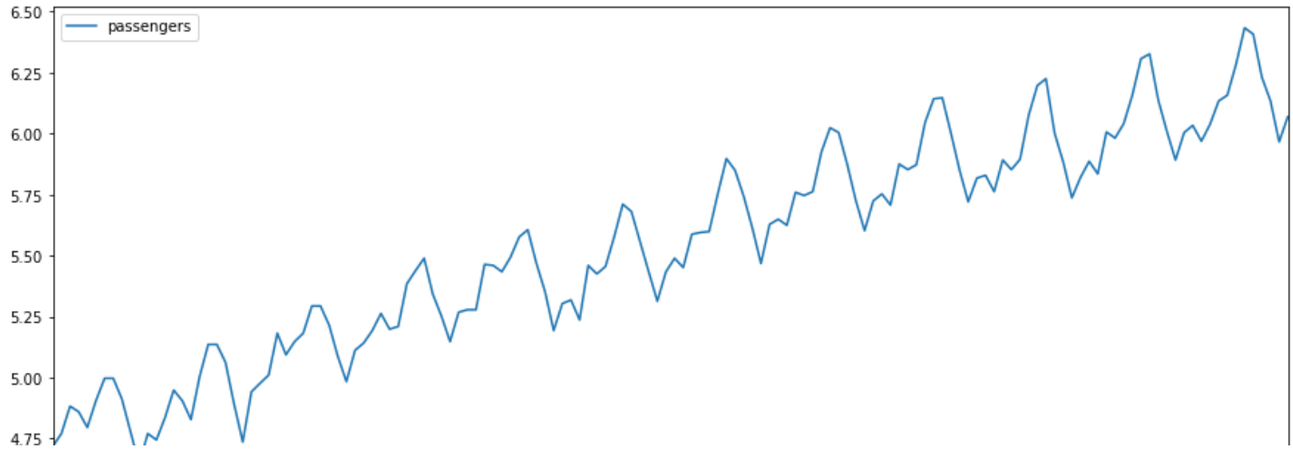
```
differenced_df = df.diff()

differenced_df.plot(figsize=(15, 6))
plt.show()
```



Nuestro dataset tiene varianza no constante, por lo que le aplicaremos logaritmo natural

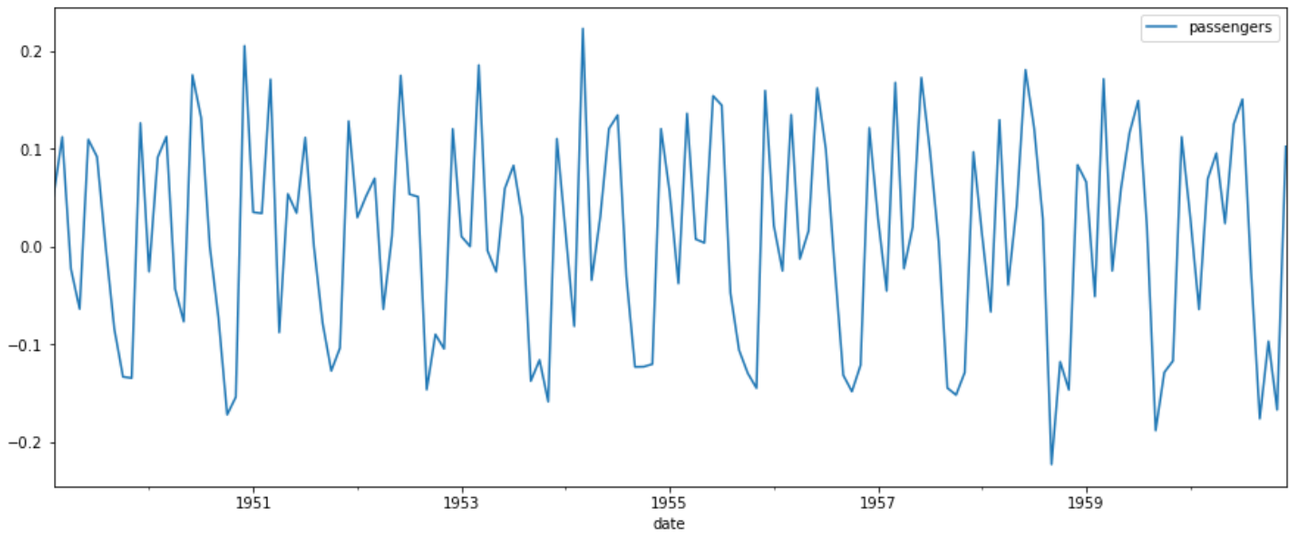
In [5]:

```
df_log = np.log(df)
df_log.plot(figsize=(15, 6))
plt.show()
```



In [6]:

```
df_log.diff().tail(-1).plot(figsize=(15, 6))
plt.show()
```

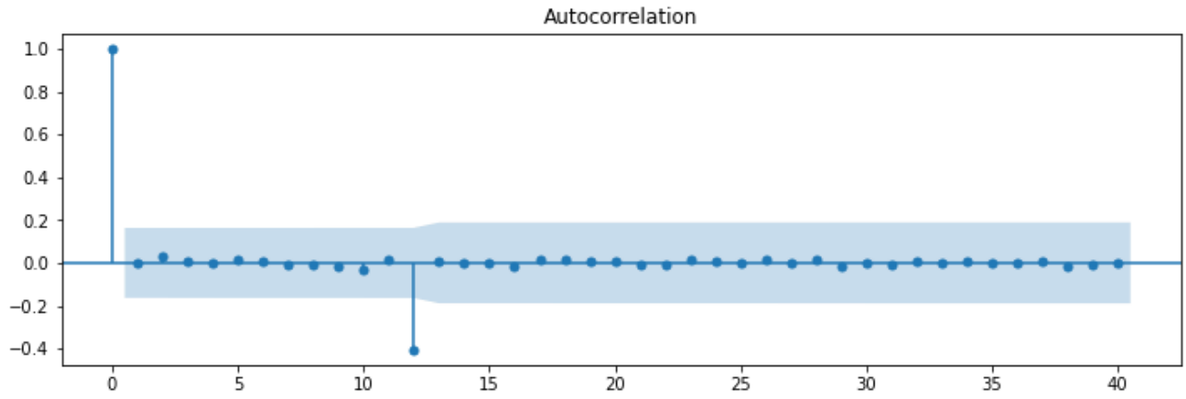


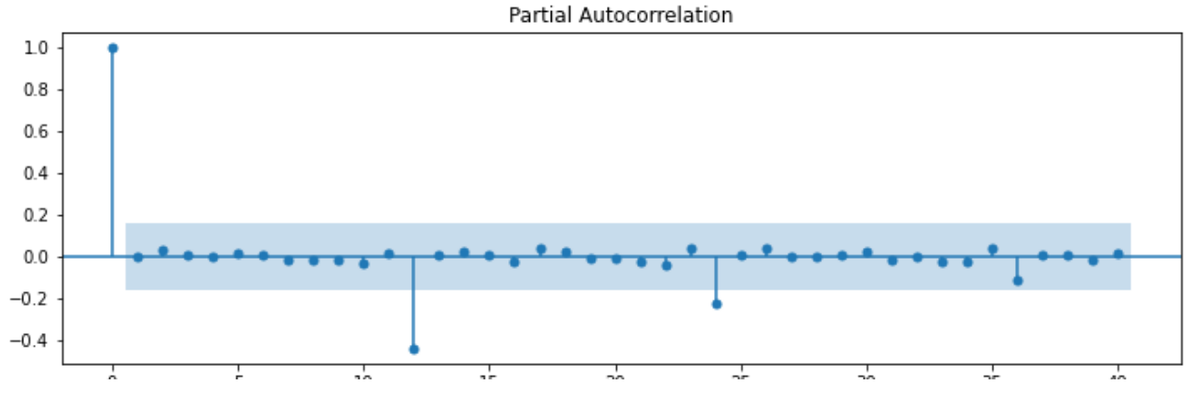
ACF y PACF

Generaremos gráficas de PACF y ACF para determinar si existe estacionalidad y el orden de p,d y q

In [30]:

```
fig = plt.figure(figsize=(12,8))
ax1 = fig.add_subplot(211)
fig = plot_acf(model.resid, lags=40, ax=ax1)
ax2 = fig.add_subplot(212)
fig = plot_pacf(model.resid, lags=40, ax=ax2)
```





0 5 10 15 20 25 30 35 40

ARIMA

Genere diferentes modelos de ARIMA modificando sus valores de p,d y q para obtener el menor AIC posible.

In [29]:

```
arima_model = SARIMAX(df_log,trend='n', order= (1,1,1),seasonal_order=(1,1,1,12))
model = arima_model.fit()
print(model.summary())
```

```
 This problem is unconstrained.
RUNNING THE L-BFGS-B CODE

           * * *

Machine precision = 2.220D-16
 N =            5     M =           10

At X0         0 variables are exactly at the bounds

At iterate    0    f= -1.66507D+00    |proj g|=  5.05749D+00

At iterate    5    f= -1.68179D+00    |proj g|=  8.43250D-01

At iterate   10    f= -1.69018D+00    |proj g|=  8.59472D-02

At iterate   15    f= -1.69035D+00    |proj g|=  2.07149D-01

At iterate   20    f= -1.69731D+00    |proj g|=  4.01666D-01

At iterate   25    f= -1.69760D+00    |proj g|=  1.63498D-02

At iterate   30    f= -1.69773D+00    |proj g|=  1.74488D-01

At iterate   35    f= -1.69813D+00    |proj g|=  2.83014D-01

At iterate   40    f= -1.70149D+00    |proj g|=  1.19817D+00

At iterate   45    f= -1.70244D+00    |proj g|=  7.01683D-03

           * * *

Tit   = total number of iterations
Tnf   = total number of function evaluations
Tnint = total number of segments explored during Cauchy searches
Skip  = number of BFGS updates skipped
Nact  = number of active bounds at final generalized Cauchy point
Projg = norm of the final projected gradient
F     = final function value

           * * *

   N    Tit     Tnf  Tnint  Skip  Nact     Projg        F
    5     47     67      1     0     0   3.188D-03  -1.702D+00
  F =  -1.7024435159419213

CONVERGENCE: REL_REDUCTION_OF_F_<=_FACTR*EPSMCH
                                     SARIMAX Results
```

```
==========================================================================================
=
Dep. Variable:                        passengers   No. Observations:                  1
44
Model:             SARIMAX(1, 1, 1)x(1, 1, 1, 12)   Log Likelihood                 245.1
52
Date:                            Mon, 06 Mar 2023   AIC                           -480.3
04
Time:                                    08:44:00   BIC                           -465.
928
Sample:                                01-01-1949   HQIC                          -474.4
62
                                     - 12-01-1960

Covariance Type:                              opg

==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.1690      0.213      0.795      0.427      -0.248       0.586
ma.L1         -0.5635      0.184     -3.058      0.002      -0.925      -0.202
ar.S.L12      -0.0986      0.197     -0.500      0.617      -0.485       0.287
ma.S.L12      -0.4979      0.210     -2.371      0.018      -0.910      -0.086
sigma2         0.0013      0.000      8.460      0.000       0.001       0.002
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                 3.49
Prob(Q):                              0.91   Prob(JB):                         0.17
Heteroskedasticity (H):               0.61   Skew:                            -0.01
Prob(H) (two-sided):                  0.11   Kurtosis:                         3.80
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

En la siguiente tabla encontrará un resumen de los resultados utilizando diferentes parámetros

Parameters	AIC
[1,1,0]	-236
[0,1,1]	-238
[1,1,1]	-242
[1,1,1] [0,1,0]	-448
[1,1,1] [1,0,0]	-467
[1,1,1] [0,0,1]	-330
[1,1,1] [1,1,0]	-475
[1,1,1] [0,1,1]	-481
[1,1,1] [1,1,1]	-480

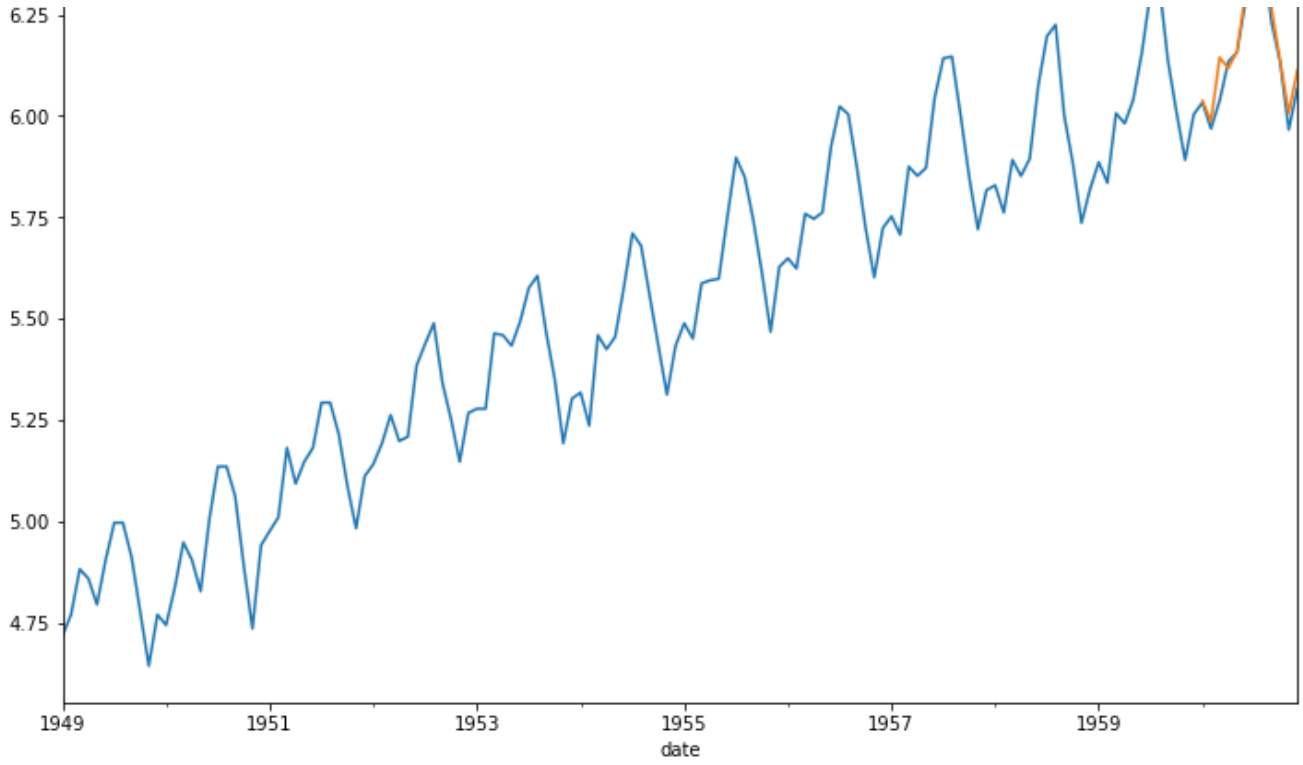
Predicciones

In [22]:

```
predictions = df_log.copy()

predictions['forecast'] = model.predict(start = 132, end= 143, dynamic= True)
predictions[['passengers', 'forecast']].plot(figsize=(12, 8))
```

Out[22]:

```
<AxesSubplot:xlabel='date'>
```

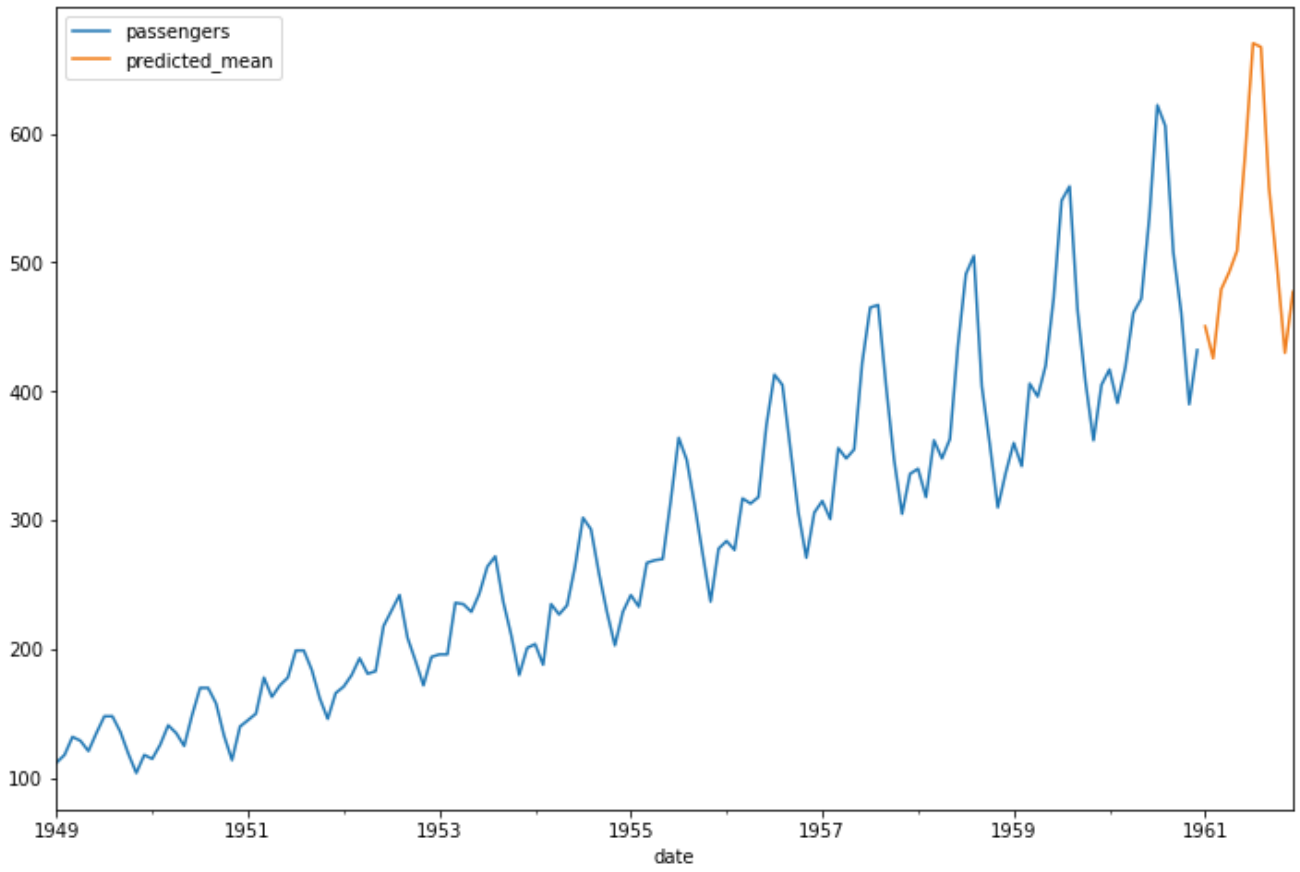


In [24]:

```
forecast = model.forecast(steps = 12)

np.exp(df_log).plot(figsize=(12, 8),legend=True)
```

Out[24]:

```
<AxesSubplot:xlabel='date'>
```



In [ ]:

```

```